

Introduction to HTML - Complete Lesson Guide

Using VS Code, Git/GitHub, and Chrome DevTools

Course Overview

Duration: 2-3 hours

Target Audience: Complete beginners to web development

Tools Required: VS Code, Git, GitHub account, Chrome browser

Learning Objectives

By the end of this lesson, students will be able to:

- Understand what HTML is and its role in web development
 - Set up a development environment with VS Code
 - Create and structure basic HTML documents
 - Use Git for version control and GitHub for hosting
 - Inspect and debug HTML using Chrome DevTools
 - Deploy a simple website to GitHub Pages
-

Prerequisites Setup (15 minutes)

Required Software Installation

1. **VS Code** - Download from code.visualstudio.com
2. **Git** - Download from git-scm.com
3. **Chrome Browser** - For DevTools
4. **GitHub Account** - Sign up at github.com

VS Code Extensions to Install

- Live Server
 - HTML CSS Support
 - Auto Rename Tag
 - Prettier - Code formatter
-

Part 1: What is HTML? (20 minutes)

Introduction to HTML

- **HTML** = HyperText Markup Language
- The skeleton/structure of web pages
- Uses tags to define elements
- Works with CSS (styling) and JavaScript (interactivity)

Basic HTML Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Webpage</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is my first paragraph.</p>
</body>
</html>
```

Key Concepts to Explain

- **Elements vs Tags:** `<p>content</p>` - element includes opening tag, content, and closing tag
- **Attributes:** Additional information about elements (`class` , `id` , `src` , etc.)
- **Nesting:** Elements inside other elements
- **Semantic HTML:** Using meaningful tags (`<header>` , `<nav>` , `<main>` , `<footer>`)

Part 2: Setting Up the Development Environment (25 minutes)

Step 1: Create Project Structure

1. Create a new folder called `my-first-website`
2. Open VS Code
3. File → Open Folder → Select your project folder

Step 2: Initialize Git Repository

```
git init
git config user.name "Your Name"
git config user.email "your.email@example.com"
```

Step 3: Create Your First HTML File

1. Create `index.html` in VS Code
2. Type `!` and press Tab (Emmet abbreviation)
3. Modify the generated HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Website</title>
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
    <nav>
      <ul>
        <li><a href="#about">About</a></li>
        <li><a href="#projects">Projects</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>Hello! I'm learning HTML and web development.</p>
      
    </section>

    <section id="projects">
      <h2>My Projects</h2>
      <div>
        <h3>Project 1</h3>
        <p>Description of my first project.</p>
      </div>
    </section>

    <section id="contact">
      <h2>Contact Me</h2>
      <form>
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
      </form>
    </section>
  </main>
</body>
</html>
```

```
<label for="message">Message:</label>
<textarea id="message" name="message" rows="4" required></textarea>

<button type="submit">Send Message</button>
</form>
</section>
</main>

<footer>
  <p>&copy; 2025 My First Website. All rights reserved.</p>
</footer>
</body>
</html>
```

Step 4: Using Live Server

1. Right-click on `index.html`
 2. Select "Open with Live Server"
 3. Your website opens in the browser with live reload
-

Part 3: Essential HTML Elements (30 minutes)

Text Elements

- `<h1>` to `<h6>` - Headings
- `<p>` - Paragraphs
- `<em>` - Emphasis (italic)
- `<strong>` - Strong importance (bold)
- `<br>` - Line break
- `<hr>` - Horizontal rule

Lists

```
<!-- Unordered List -->
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>

<!-- Ordered List -->
<ol>
  <li>First step</li>
  <li>Second step</li>
  <li>Third step</li>
</ol>
```

Links and Images

```
<!-- Links -->
<a href="https://www.example.com">External link</a>
<a href="#section1">Internal link</a>
<a href="mailto:someone@example.com">Email link</a>

<!-- Images -->

```

Forms (Interactive Elements)

```
<form>
  <input type="text" placeholder="Enter text">
  <input type="email" placeholder="Enter email">
  <input type="password" placeholder="Enter password">
  <select>
    <option>Option 1</option>
    <option>Option 2</option>
  </select>
  <textarea placeholder="Enter message"></textarea>
  <button type="submit">Submit</button>
</form>
```

Part 4: Chrome DevTools Deep Dive (25 minutes)

Opening DevTools

- Right-click → Inspect Element
- F12 key
- Ctrl+Shift+I (Windows) / Cmd+Option+I (Mac)

Elements Panel

1. **HTML Structure:** Navigate through the DOM tree
2. **Live Editing:**
 - Right-click element → Edit as HTML
 - Double-click on text to edit
 - Changes are temporary (refresh to reset)

Key DevTools Features to Demonstrate

1. **Inspect Element:** Hover over code to highlight page elements
2. **Console:** View errors and warnings
3. **Network Tab:** See which files are loading
4. **Responsive Design Mode:** Test different screen sizes
5. **Accessibility:** Check color contrast and screen reader compatibility

Practical Exercise

Have students:

1. Inspect different elements on their page
 2. Temporarily change text content
 3. Add/remove CSS classes (if any are present)
 4. Check for console errors
 5. Test responsive design
-

Part 5: Git Version Control (20 minutes)

Basic Git Workflow

```
# Check status
git status

# Add files to staging
git add index.html
# or add all files
git add .

# Commit changes
git commit -m "Add initial HTML structure"

# View commit history
git log --oneline
```

Common Git Commands to Teach

- `git status` - Check current state
- `git add` - Stage changes
- `git commit` - Save changes with message
- `git log` - View history
- `git diff` - See changes

Best Practices

- Write clear, descriptive commit messages
 - Commit frequently with logical chunks of work
 - Use present tense: "Add navigation menu" not "Added navigation menu"
-

Part 6: GitHub Integration (25 minutes)

Creating a GitHub Repository

1. Go to [GitHub.com](https://github.com)

2. Click "New repository"
3. Name it `my-first-website`
4. Keep it public
5. Don't initialize with README (we already have files)

Connecting Local Repository to GitHub

```
# Add remote origin
git remote add origin https://github.com/yourusername/my-first-website.git

# Push to GitHub
git branch -M main
git push -u origin main
```

GitHub Pages Setup

1. Go to repository Settings
 2. Scroll to "Pages" section
 3. Select "Deploy from a branch"
 4. Choose "main" branch
 5. Click Save
 6. Your site will be available at: `https://yourusername.github.io/my-first-website`
-

Part 7: Hands-On Practice Project (30 minutes)

Student Exercise: Personal Portfolio

Students will create their own version with:

1. **Header** with name and navigation
2. **About section** with personal information
3. **Skills section** with a list of interests/skills
4. **Projects section** (can be placeholder for now)
5. **Contact section** with a simple form
6. **Footer** with copyright information

Requirements:

- Use semantic HTML tags
 - Include at least one image
 - Create a navigation menu with anchor links
 - Use different heading levels appropriately
 - Include a form with various input types
 - Commit changes to Git
 - Push to GitHub
 - Deploy to GitHub Pages
-

Part 8: Testing and Validation (15 minutes)

HTML Validation

- Use [W3C Markup Validator](#)
- Paste code or upload file to check for errors

Accessibility Basics

- Always include `alt` attributes for images
- Use semantic HTML elements
- Ensure proper heading hierarchy
- Test with keyboard navigation (Tab key)

Cross-Browser Testing

- Test in different browsers (Chrome, Firefox, Safari, Edge)
 - Use DevTools responsive mode for mobile testing
-

Assessment and Next Steps

Quick Assessment Questions

1. What does HTML stand for?

2. What is the difference between an element and a tag?
3. How do you create a link to another section of the same page?
4. What Git command do you use to save your changes?
5. How can you inspect an element on a webpage?

Project Submission

Students should submit:

1. GitHub repository URL
2. Live GitHub Pages URL
3. Screenshot of their website
4. Brief reflection on what they learned

Next Steps

- **CSS:** Styling and layout
 - **JavaScript:** Interactivity and dynamic content
 - **Responsive Design:** Mobile-first approach
 - **Web Accessibility:** Making sites usable for everyone
 - **Modern Development:** Frameworks like React, Vue, or Angular
-

Common Troubleshooting

Git Issues

- **Authentication:** Use personal access tokens instead of passwords
- **Remote exists:** Use `git remote set-url origin <new-url>`
- **Branch naming:** GitHub now uses `main` instead of `master`

HTML Issues

- **Missing closing tags:** Use VS Code's auto-completion
- **Broken links:** Check file paths and spelling
- **Images not loading:** Verify image paths and file extensions

DevTools Tips

- **Element not selectable:** Try pausing JavaScript execution
 - **Changes disappearing:** Remember DevTools changes are temporary
 - **Console errors:** Read error messages carefully for clues
-

Additional Resources

Documentation

- [MDN HTML Documentation](#)
- [W3Schools HTML Tutorial](#)
- [Git Documentation](#)

Practice Websites

- [freeCodeCamp](#)
- [Codecademy HTML Course](#)
- [The Odin Project](#)

Tools and Extensions

- [VS Code HTML Snippets](#)
 - [Can I Use](#) - Browser compatibility checker
 - [HTML5 Boilerplate](#) - Professional front-end template
-

Lesson Wrap-Up

Key Takeaways

1. HTML provides the structure and content of web pages
2. VS Code with extensions makes HTML development easier
3. Git tracks changes and enables collaboration
4. GitHub hosts code and provides free website hosting
5. Chrome DevTools are essential for debugging and testing

6. Semantic HTML improves accessibility and SEO

Homework Assignment

1. Add more content to your personal website
2. Create a second page and link to it
3. Experiment with different HTML elements not covered in class
4. Validate your HTML and fix any errors
5. Share your GitHub Pages URL with friends and family

This lesson guide provides a comprehensive introduction to HTML development using modern tools and best practices. Adjust timing and content based on your students' pace and prior experience.